UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Answers are unofficial reference answers, not official UTM marking schemes.

2. Marking points show the expected answer and mark allocation; check them against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Approved real questions only; no generators, placeholders or parametric variants. Questions and answers are for local study use only.

5. Structure derived from the supplied CSC429 papers; every question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 10 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION 1 (5 marks)

Source: CSC429-2014-12, page [2]

Unofficial model answer:

Von Neumann architecture: The classic stored-program computer model in which a single memory holds both instructions and data, and the CPU fetches instructions and operands over a shared bus. A correct drawing shows five blocks: (1) Input unit, (2) Memory unit, (3) Arithmetic Logic Unit (ALU), (4) Control Unit, and (5) Output unit, with the ALU and Control Unit grouped as the CPU (Central Processing Unit). Arrows should show the flow: input → memory; memory ↔ CPU (address bus, data bus, control bus); CPU → output. The Control Unit fetches and decodes instructions and generates control signals; the ALU performs arithmetic and logic operations; memory stores both program and data; input/output units transfer data to and from the outside world. Example for each component: Input unit — keyboard, mouse, scanner; Memory unit — RAM (main memory), ROM/cache; ALU — performs addition (e.g., ADD R1,R2) or logical AND/OR/comparison; Control unit — the program counter and instruction decoder that sequence fetch-decode-execute; Output unit — monitor/display, printer, speaker. (A labelled block diagram is required in the exam; the text above gives the blocks and connections that must appear.)

Marking points:

• *1 mark: Correct block diagram showing the five units — input, memory, ALU, control unit, output — with CPU grouping of ALU + control unit.*

• *1 mark: Correct data/control flow arrows (input→memory, memory↔CPU via buses, CPU→output) and shared memory for instructions and data.*

• *1 mark: Appropriate example for input and output units (e.g., keyboard/scanner; monitor/printer).*

• *1 mark: Appropriate example for memory unit and ALU (e.g., RAM/ROM; addition/logic operation).*

• *1 mark: Appropriate example/role for control unit (e.g., PC, instruction decoder, fetch-decode-execute sequencing).*

Model confidence: high.

Model note: No figure was supplied; the answer describes the standard Von Neumann block diagram that the student must draw. Marking assumes a 5-mark allocation with roughly one mark per component/example plus diagram correctness; exact mark split may vary.

QUESTION 2 (5 marks)

Source: CSC429-2015-12, page [2]

Unofficial model answer:

a) Fourth generation (VLSI / microprocessors). The production of personal computers is the main feature of the fourth generation; prices were the lowest compared with earlier generations, and VLSI microprocessors gave fast processing with less power consumption.

b) Fifth generation. Machines that emulate human intelligence and can process and respond to natural language (AI-oriented systems) are developed in the fifth generation.

c) Third generation. In the third generation (integrated circuits), air conditioning is no longer required because the heat produced by the machine is reduced; however, a fan is still needed for heat discharging.

d) Third generation. The integrated circuit, comprising a number of transistors, resistors and capacitors, was invented by Jack Kilby in the third generation.

e) Second generation. Assembly language was introduced during the second generation to replace machine language for more reliable and improved performance.

Marking points:

© Hak Cipta Universiti Teknologi MARA

• 1 mark: (a) Fourth generation (VLSI/microprocessor) — personal computers, lowest price, fast processing with less power consumption.

• 1 mark: (b) Fifth generation — machines emulating human intelligence and processing/responding to natural language.

• 1 mark: (c) Third generation — no air conditioning needed, reduced heat, fan still required for heat discharge.

• 1 mark: (d) Third generation — integrated circuit invented by Jack Kilby.

• 1 mark: (e) Second generation — assembly language introduced to replace machine language.

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. Generation numbering follows the standard CSC429 five-generation taxonomy (1st vacuum tubes, 2nd transistors, 3rd integrated circuits, 4th VLSI/microprocessors, 5th AI). No figure or table was supplied.

QUESTION 3 (5 marks)

Source: CSC429-2023-02, page [2]

Unofficial model answer:

The question text is garbled by OCR, so the intended operands are reconstructed from context: (a) is a binary subtraction $1.00001001_2 - 0.00110001_2$ (the 'OOH' is read as 0011 0001), and (b) is a hexadecimal subtraction $0.FA3_{16} - 0.651_{16}$ (the 'i6' is read as subscript 16 and '651s' as 651_{16}). Both are fractional subtractions, so the workflow is: align radix points, pad with zeros, subtract column by column with borrows, then state the result.

(a) Binary subtraction $1.00001001_2 - 0.00110001_2$. Align both numbers to 8 fractional bits: minuend 1.00001001, subtrahend 0.00110001. Working from the least significant bit (rightmost): bit 8: 1−1=0; bit 7: 0−0=0; bit 6: 0−0=0; bit 5: 0−1 requires a borrow — borrow from bit 4, giving 10_2−1=1, and bit 4 becomes 0; bit 4: now 0−1 again requires a borrow from bit 3, giving 10_2−1=1, bit 3 becomes 0; bit 3: 0−1 borrow from bit 2, giving 1, bit 2 becomes 0; bit 2: 0−1 borrow from bit 1, giving 1, bit 1 becomes 0; bit 1: 0−0=0; integer bit: 1−0=1. Result: 1.00010000_2, i.e. 1.0001_2 (decimal check: 1.03515625 − 0.19140625 = 0.84375 = 1.0001_2 is not equal; the correct decimal check is 1.00001001_2 = 1.03515625 and 0.00110001_2 = 0.19140625, difference 0.84375 = 0.11011_2, so the borrow chain above must be rechecked). Corrected column-by-column working: 1.00001001 − 0.00110001. Bit 8: 1−1=0. Bit 7: 0−0=0. Bit 6: 0−0=0. Bit 5: 0−1: borrow from bit 4 (which is 0, so borrow propagates to bit 3, bit 2, bit 1, then the integer 1). The integer 1 becomes 0, bit 1 becomes 10_2, lends 1 to bit 2 leaving 1, bit 2 lends to bit 3 leaving 1, bit 3 lends to bit 4 leaving 1, bit 4 lends to bit 5 leaving 1, and bit 5 becomes 10_2. Then bit 5: 10−1=1; bit 4: 1−1=0; bit 3: 1−0=1; bit 2: 1−0=1; bit 1: 1−0=1; integer: 0−0=0. Result: $0.11011000_2 = 0.11011_2 = 0.84375_{10}$, which matches the decimal check. So the answer is 0.11011_2 (0.11011000_2).

(b) Hexadecimal subtraction $0.FA3_{16} - 0.651_{16}$. Align to three hex digits: FA3 − 651. Least significant digit: 3−1=2. Middle digit: A−5 = 10−5 = 5. Most significant digit: F−6 = 15−6 = 9. Result: 0.952_{16}. Decimal check: $0.FA3_{16}$ = 15/16 + 10/256 + 3/4096 = 0.9375 + 0.0390625 + 0.000732421875 = 0.977294921875; 0.651_{16} = 6/16 + 5/256 + 1/4096 = 0.375 + 0.01953125 + 0.000244140625 = 0.394775390625; difference = 0.58251953125 = 9/16 + 5/256 + 2/4096 = 0.5625 + 0.01953125 + 0.00048828125 = 0.58251953125, confirming 0.952_{16}.

Marking points:

• 1 mark: correct alignment of radix points and zero-padding for (a)

• 1 mark: correct borrow-propagating subtraction workflow for (a) with result 0.11011_2 (accept 0.11011000_2)

• 1 mark: correct digit-by-digit hexadecimal subtraction for (b) with result 0.952_{16}

• 1 mark: correct handling of the hex letters (A=10, F=15) and no spurious borrow in (b)

• 1 mark: verification by decimal conversion (or equivalent check) for at least one part, with clear workflow shown

Model confidence: medium.

© Hak Cipta Universiti Teknologi MARA

Model note: The printed question is corrupted by OCR: '1.0000 10012-0.0011 OOH2' and 'O.FA3i6-0.651s' are ambiguous. I assumed (a) is $1.00001001_2 - 0.00110001_2$ and (b) is $0.FA3_{16} - 0.651_{16}$, based on the '2' and '16' subscripts and the hex letters F, A. If the intended operands differ (e.g. 0.00110011_2 or 0.651_8), the numerical results change; the method and marking allocation remain valid. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 4 (5 marks)

Source: CSC429-2017-07, page [2]

Unofficial model answer:

a) The X machine is the Atanasoff–Berry Computer (ABC), the first automatic electronic digital computer to use the binary system. It was a special-purpose machine for solving systems of linear equations, using 30 arithmetic/logic units that could add and subtract, a rotating drum memory holding thirty binary numbers of fifty digits each, and input/output components. [2 marks]

b) The inventor is John Vincent Atanasoff, who built it with his graduate assistant Clifford Berry at Iowa State College (1939–1942). [1 mark]

c) Based on Figure 1, the input component is the card mechanism (punch-card reader/mechanism) used to enter data, and the output component is the meter (a meter/display) used to read results; the figure also shows the drum, brushes, gear box, power supply regulator, and circuit with dual triodes as internal/auxiliary parts. [2 marks]

Marking points:

- *1 mark: identifies X machine as the Atanasoff–Berry Computer (ABC)*
- *1 mark: states it is the first automatic electronic digital computer using binary / special-purpose linear-equation solver*
- *1 mark: names John Vincent Atanasoff (accept Atanasoff–Berry / with Clifford Berry)*
- *1 mark: identifies input component as card mechanism / punch-card reader*
- *1 mark: identifies output component as meter*

Model confidence: medium.

Model note: Figure 1 is supplied only as a cropped image with garbled OCR text; input/output identification is inferred from the visible labels 'card mechanism' and 'meter'. The question's '30 arithmetic/logic unit' and 'thirty binary numbers of fifty digits' are taken as given. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 5 (10 marks)

Source: CSC429-2016-12, page [5]

Unofficial model answer:

a) Three main key factors for memory selection: (1) Cost per bit: Memory technology is chosen to balance cost against capacity; SRAM is fast but expensive per bit, DRAM is cheaper per bit, and magnetic/optical storage is cheapest per bit, so a memory hierarchy uses expensive fast memory for cache and cheaper slower memory for main memory and secondary storage. (2) Access time (latency): The time to read/write a memory location affects overall system performance; faster access time is needed for cache and registers, while slower access time is acceptable for main memory and disk. (3) Capacity: The amount of memory required to hold the working set and programs; larger capacity generally increases cost and may increase access time, so capacity is selected to meet application and OS requirements. Other acceptable factors include volatility, erasability, and physical size/form factor, but the three above are the main ones.

b) Two examples of internal cabling memory: (i) registers (CPU registers), (ii) cache memory (L1/L2/L3). Two examples of system memory: (i) main memory (RAM, e.g., DRAM), (ii) secondary storage (hard disk drive, solid-state drive, or ROM). Note: 'internal cabling memory' is interpreted as memory internal to the CPU/system that is directly wired/connected, i.e., registers and cache; 'system memory' refers to memory used by the system for programs and data, i.e., main memory and secondary storage.

© Hak Cipta Universiti Teknologi MARA

Marking points:

• 1 mark: Correctly identifies cost per bit as a key factor and briefly explains the trade-off between cost and speed/capacity.

• 1 mark: Correctly identifies access time (latency) as a key factor and explains its impact on performance.

• 1 mark: Correctly identifies capacity as a key factor and explains the need to match application/OS requirements.

• 1 mark: Additional credit for mentioning other valid factors (e.g., volatility, erasability, physical size) up to the 3-factor limit.

• 1 mark: Gives one correct example of internal cabling memory (e.g., registers).

• 1 mark: Gives a second correct example of internal cabling memory (e.g., cache).

• 1 mark: Gives one correct example of system memory (e.g., main memory/RAM).

• 1 mark: Gives a second correct example of system memory (e.g., secondary storage/ROM).

• 1 mark: Clarity and correct use of terminology in part (a).

• 1 mark: Clarity and correct categorization in part (b).

Model confidence: medium.

Model note: The term 'internal cabling memory' is non-standard; it is assumed to mean memory internal to the CPU/system that is directly connected (registers and cache). If the examiner intended a different meaning, part (b) examples may vary. The answer is based on standard computer organization textbooks (e.g., Stallings, Patterson & Hennessy).

QUESTION 6 (10 marks)

Source: CSC429-2015-12, page [6]

Unofficial model answer:

a) Examples for each classification of multiple processor organization: (i) Symmetric Multiprocessors (SMP): a dual-socket Intel Xeon server board with two identical CPUs sharing a single main memory and I/O subsystem through a common system bus or crossbar; (ii) Clusters: a Beowulf cluster of commodity PCs connected by an Ethernet/Gigabit network, each node running its own OS and cooperating as a single system; (iii) Non-Uniform Memory Access (NUMA): a modern multi-socket server (e.g., AMD EPYC or Intel Xeon Scalable) where each processor has local memory and accesses remote memory attached to other processors with higher latency. (Other valid examples: SMP = multicore processor with shared L3; Clusters = Hadoop cluster; NUMA = SGI Origin 2000.)

b) Simplified architectures (described textually because diagrams cannot be drawn here):

i. Symmetric Multiprocessors (SMP): A single shared main memory is connected via a system bus (or crossbar switch) to multiple identical processors (P1, P2, ..., Pn). Each processor also connects to the same bus for I/O devices. All processors have equal access to all memory and I/O, and a single OS manages the whole system. Diagram: [P1][P2]...[Pn] each attached to a common bus; bus attached to shared memory and I/O.

ii. Clusters: Multiple independent computer nodes (each node = processor + local memory + local I/O + OS) interconnected by a high-speed network (e.g., Ethernet, InfiniBand) through network interfaces. No shared global memory; communication via message passing. Diagram: several node boxes, each containing CPU, memory, I/O, connected to a central network switch/hub.

iii. NUMA: Multiple processors (or processor groups) each with its own local memory, interconnected by a scalable interconnection network (e.g., crossbar or mesh). A processor accesses its local memory directly (low latency) and remote memory through the interconnection network (higher latency). I/O devices may be attached to some nodes. Diagram: [P1 + local memory] --- interconnection network --- [P2 + local memory] --- ... with I/O attached to nodes.

Marking points:

• 1 mark: Correct example for Symmetric Multiprocessors (e.g., dual-socket server with shared memory).

• 1 mark: Correct example for Clusters (e.g., Beowulf cluster of PCs with network).

• 1 mark: Correct example for NUMA (e.g., multi-socket server with local/remote memory).

© Hak Cipta Universiti Teknologi MARA

• 1 mark: SMP diagram shows multiple identical processors connected to a common bus/shared memory.

• 1 mark: SMP diagram indicates equal access to shared memory and I/O.

• 1 mark: Clusters diagram shows multiple independent nodes (each with CPU, memory, I/O) connected via a network.

• 1 mark: Clusters diagram indicates no shared memory (message passing).

• 1 mark: NUMA diagram shows processors with local memory and an interconnection network.

• 1 mark: NUMA diagram distinguishes local vs. remote memory access (different latency).

• 1 mark: Overall clarity and correct labeling of components in all three diagrams.

Model confidence: high.

Model note: The question asks for drawings; since diagrams cannot be produced in this text format, the answer describes the required architectures in words. The marking scheme assumes the examiner awards marks for the presence and correctness of key components in each diagram. Examples given are illustrative; other valid examples should be accepted.

QUESTION 7 (10 marks)

Source: CSC429-2026-02, page [5]

Unofficial model answer:

a) An I/O module is installed when a computer system needs to connect to and communicate with external devices (peripherals) such as keyboards, displays, disks, or network adapters. It is required whenever the CPU must transfer data to or from a device that is not directly connected to the system bus, providing the interface, control, and data buffering needed for the exchange.

b) If the I/O module is not installed, the CPU cannot communicate with external devices. The system would be unable to perform input or output operations, meaning no data could be read from or written to peripherals; the computer would be isolated and unable to interact with the outside world, severely limiting its usefulness.

c) One category of I/O module is the 'human-readable' or 'user-interface' module. This category includes devices that allow humans to interact with the computer, such as keyboards, mice, monitors, and printers. These modules convert between the computer's internal binary representation and forms that humans can perceive or manipulate (e.g., text on a screen, printed characters, or keystrokes).

d) 'Hot connection' (hot plugging) refers to the ability to connect or disconnect a device to or from a computer system while the system is powered on and running, without causing damage or requiring a reboot. The system automatically detects and configures the device, allowing immediate use.

e) Three common interfaces to external devices are: (1) USB (Universal Serial Bus), (2) HDMI (High-Definition Multimedia Interface), and (3) Ethernet (for network connections). Other acceptable examples include VGA, Thunderbolt, SATA, and Bluetooth.

Marking points:

• 1 mark: (a) Correctly states that an I/O module is installed when the system needs to connect to external devices/peripherals.

• 1 mark: (b) Explains that without the I/O module, the CPU cannot communicate with external devices, so input/output operations are impossible.

• 2 marks: (c) Identifies one category of I/O module (e.g., human-readable, machine-readable, communication) and explains its function with examples.

• 2 marks: (d) Defines 'hot connection' as connecting/disconnecting a device while the system is powered on, with automatic detection/configuration.

• 3 marks: (e) Lists three correct common interfaces to external devices (1 mark each).

• 1 mark: Overall clarity and correct use of terminology.

Model confidence: high.

Model note: The question does not refer to any figure or table. The answer assumes standard computer organization definitions. For part (c), any valid category is acceptable; the example given is human-readable. For part (e), many interfaces are valid; the marking should accept any three correct ones.

© Hak Cipta Universiti Teknologi MARA

QUESTION 8 (10 marks)

Source: CSC429-2019-12, page [4]

Unofficial model answer:

a) Two different types of bus: (1) Address bus – carries the memory or I/O address from the processor to memory/peripherals; it is unidirectional. (2) Data bus – carries data (and instructions) between the processor, memory and I/O devices; it is bidirectional. (Other acceptable examples: control bus, system bus, expansion bus.)

b) Given word size = 32 bits and block size = 256 bits. (i) Byte for word: 32 bits / 8 bits per byte = 4 bytes per word. (ii) Byte for block: 256 bits / 8 = 32 bytes per block. (iii) Size of the register: the register that holds one word must be at least one word wide, so register size = 32 bits (4 bytes). (iv) Size of the data bus: the data bus must transfer one word per bus cycle, so data bus width = 32 bits (4 bytes).

Marking points:

- *1 mark: correct identification of address bus (unidirectional, carries addresses)*
- *1 mark: correct identification of data bus (bidirectional, carries data) OR another valid bus type such as control bus*
- *1 mark: byte for word = 32/8 = 4 bytes*
- *1 mark: byte for block = 256/8 = 32 bytes*
- *1 mark: register size = 32 bits (or 4 bytes) with justification that it holds one word*
- *1 mark: data bus size = 32 bits (or 4 bytes) with justification that it transfers one word per cycle*
- *4 marks: correct working/units shown for each of the four calculations (1 mark each)*

Model confidence: high.

Model note: This is an unofficial model answer, not an official UTM marking scheme. The question does not specify whether 'size of the register' refers to a general-purpose register or a memory data register; the answer assumes a register that holds one word. Similarly, the data bus width is assumed to match the word size, which is the standard assumption when no other information is given.

QUESTION 9 (10 marks)

Source: CSC429-2018-01, page [5]

Unofficial model answer:

a) Two types of interrupt: (1) Maskable interrupt — an interrupt request that can be temporarily disabled (masked) by the CPU, typically via an interrupt-enable flag or interrupt mask register; it is used for ordinary I/O devices whose service can be delayed, e.g., a keyboard or network controller asserting IRQ. (2) Non-maskable interrupt (NMI) — an interrupt that cannot be disabled by software and is reserved for critical events such as power failure, memory parity error, or hardware fault; the CPU always takes it, often transferring control to a dedicated handler. (Other acceptable pairs: external vs internal, hardware vs software, vectored vs non-vectored.)

b) Three interfaces to external devices: (1) Programmed I/O (polling) — the CPU repeatedly tests a device status register and transfers data through the device's data register; simple but wastes CPU cycles. (2) Interrupt-driven I/O — the device signals the CPU with an interrupt when it is ready or has completed a transfer, so the CPU can do other work until then; the interrupt handler services the device. (3) Direct Memory Access (DMA) — a DMA controller takes over the bus and transfers blocks of data directly between the device and main memory without per-word CPU involvement, interrupting the CPU only at completion; suited to high-speed devices such as disks. (Also acceptable: memory-mapped I/O vs isolated/port-mapped I/O, or standard bus interfaces such as USB, SCSI, PCIe.)

Marking points:

- *1 mark: correct identification of a maskable interrupt*
- *1 mark: correct explanation of maskable interrupt (can be disabled/masked, used for ordinary I/O)*
- *1 mark: correct identification of a non-maskable interrupt (NMI)*

© Hak Cipta Universiti Teknologi MARA

• 1 mark: correct explanation of NMI (cannot be masked, used for critical events)

• 1 mark: correct identification of first interface (e.g., programmed I/O/polling)

• 1 mark: correct explanation of first interface

• 1 mark: correct identification of second interface (e.g., interrupt-driven I/O)

• 1 mark: correct explanation of second interface

• 1 mark: correct identification of third interface (e.g., DMA)

• 1 mark: correct explanation of third interface

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. The question text does not specify a particular figure or table, so generic computer-organization terminology is used. Accept equivalent alternative interrupt classifications and I/O interface examples if correctly explained.

QUESTION 10 (10 marks)

Source: CSC429-2018-01, page [5]

Unofficial model answer:

a) The addressing mode in Figure 2 is **register indirect addressing**. The instruction specifies a register (in the register field) that holds the effective address of the operand, rather than the operand itself. The operand is fetched from memory at the address contained in that register.

b) The register involved is a **general-purpose register** (e.g., R1, R2, etc.) that contains the memory address of the operand. The specific register number is given in the instruction's register field. In many architectures, any general-purpose register can be used for register indirect addressing.

c) If the register in Figure 2 is replaced with a memory location, the new addressing mode is **memory indirect addressing** (also called indirect addressing). In this mode, the instruction specifies a memory address (via the address field) that contains the effective address of the operand. Thus, the operand is accessed through two memory references: one to fetch the effective address and one to fetch the operand.

d) **Register indirect addressing executes faster** than memory indirect addressing. Justification: Register indirect addressing requires only one memory access to fetch the operand (after the effective address is already in the register). Memory indirect addressing requires at least two memory accesses: one to read the effective address from memory and another to read the operand. Since memory accesses are much slower than register accesses, register indirect addressing is significantly faster. Additionally, registers are located within the CPU, so accessing them incurs no memory latency, whereas memory indirect involves the slower main memory system.

Marking points:

• 1 mark: (a) Correctly identifies register indirect addressing.

• 1 mark: (b) States that a general-purpose register is involved (or names a specific register if shown in figure).

• 1 mark: (c) Correctly identifies memory indirect addressing (or indirect addressing).

• 2 marks: (d) States register indirect is faster (1 mark) and provides justification: fewer memory accesses (1 mark).

• 2 marks: (d) Additional justification: registers are faster than memory (1 mark) and clear comparison of access counts (1 mark).

• 3 marks: Overall clarity, correct terminology, and completeness of discussion in (d).

Model confidence: medium.

Model note: Figure 2 is described only by its caption; the actual diagram is not available. The answer assumes the figure shows an instruction with an opcode, address field, register field, and operand, typical of register indirect addressing. If the figure depicts a different format, the answer may need adjustment. The marking scheme is unofficial and based on typical CSC429 expectations.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION 11 (20 marks)

Source: CSC429-2026-02, page [6]

Unofficial model answer:

Interpretation of Figure 5: The expression is read as z = A + (C²/B)·D − 3A, i.e. z = A + ((C*C)/B)*D − 3*A. Registers A, B, C, D are fixed (read-only operands); x, y, z are scratch; the final result must end in z. Note: the OCR text 'z-A+c2/B*D-3A' is ambiguous, so the expression above is the assumed reading (see notes).

(a) Zero-address (stack) program. All operands must be pushed onto the stack; results are left on the stack and finally popped into z. Using the stack, compute C*C, divide by B, multiply by D, then add A and subtract 3*A:

```
PUSH C
PUSH C
MUL
PUSH B
DIV
PUSH D
MUL
PUSH A
ADD
PUSH A
PUSH A
ADD
PUSH A
ADD
SUB
POP z
```

(Equivalently, the 3A term can be formed as PUSH A / PUSH A / ADD / PUSH A / ADD, then SUB.) The stack holds the running value; the last POP z stores the final answer in z.

(b) One-address program. One operand is the accumulator (implicit) and the other is a named register; results go to the accumulator. Use x and y as temporaries:

```
LOAD C
MUL C
STOR x
LOAD x
DIV B
STOR x
LOAD x
MUL D
STOR x
LOAD A
ADD x
STOR y
LOAD A
MUL A
ADD A
ADD A
```

© Hak Cipta Universiti Teknologi MARA

STOR x
LOAD y
SUB x
STOR z

Here x holds the C²/B·D term, y holds A + that term, and x is reused to hold 3A; the final SUB leaves the answer in the accumulator, which is stored to z.

(c) Three-address program. Each instruction names two sources and one destination, so the expression is built directly with temporaries x and y:

MUL (C, C, x)
DIV (x, B, x)
MUL (x, D, x)
ADD (A, x, y)
MUL (A, A, x)
ADD (x, A, x)
ADD (x, A, x)
SUB (y, x, z)

This computes x = C²/B·D, y = A + x, x = 3A, and finally z = y − x = A + C²/B·D − 3A, with the answer in z as required.

Marking points:

• *2 marks: correct interpretation of Figure 5 as z = A + (C*C)/B*D - 3*A (or clearly stated equivalent)*

• *6 marks: zero-address program — correct PUSH/POP usage, correct operator order (MUL, DIV, MUL, ADD, SUB), final POP z (2 marks per logical stage)*

• *6 marks: one-address program — correct LOAD/STOR with accumulator, correct use of temporaries x and y, correct final STOR z*

• *4 marks: three-address program — correct ADD/SUB/MUL/DIV (p,q,r) form, correct temporaries, final result in z*

• *2 marks: overall correctness — final answer stored in z in all three versions and fixed registers A, B, C, D never overwritten*

Model confidence: medium.

*Model note: Figure 5 is supplied only as OCR text 'z-A+c2/B*D-3A'; the expression was interpreted as z = A + C^2/B*D - 3A. If the intended expression differs (e.g. z = A + C^2/(B*D) - 3A or z = A + C^2/B*D - 3*A with different grouping), the instruction sequences must be re-ordered accordingly. Table 1 gives only the instruction mnemonics; operand conventions (e.g. whether one-address ADD p means ACC <- ACC + p) are assumed standard. This is an unofficial model answer, not an official UTM marking scheme.*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA